

Flutter + REST API + MySQL

Complete Step-by-Step Guide — From API Setup to PM2

All Steps at a Glance

Step	Task	App / Tool
1	Start XAMPP	XAMPP
2	Create Database	Browser (phpMyAdmin)
3	Create API Folder	VS Code
4	Install Packages	Terminal
5	Write Code in Files	VS Code
6	Run Server with PM2	Terminal
7	Test in Browser	Browser
8	Connect to Flutter	VS Code

STEP 1

Start XAMPP

App: XAMPP

Open XAMPP and click Start for both:

```
Apache -> Start
MySQL -> Start
```

STEP 2

Create Database

App: Browser - phpMyAdmin

Go to: <http://localhost/phpmyadmin>

Click New -> Database name: myapp_db -> Create

Then go to SQL Tab and paste this:

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100) UNIQUE,
  password VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Click Go — Table will be created!

STEP 3

Create API Folder & Structure

App: VS Code

Create a folder named myapp_api on Desktop

VS Code -> File -> Open Folder -> myapp_api

Create this folder/file structure:

```
myapp_api/
|- config/
|   db.js
|- controllers/
```

```
|   authController.js
|- routes/
|   auth.js
|- .env
auth.js
app.js
```

STEP 4

Install Packages

App: Terminal

Open Terminal in VS Code (Terminal -> New Terminal), then run:

```
npm init -y
npm install express mysql2 bcryptjs jsonwebtoken cors dotenv
npm install -g pm2
```

STEP 5

Write Code in All Files

App: VS Code

File 1: .env

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=
DB_NAME=myapp_db
JWT_SECRET=mysecretkey123
PORT=3000
```

File 2: config/db.js

```
const mysql = require('mysql2');
require('dotenv').config();

const db = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
});

module.exports = db.promise();
```

File 3: controllers/authController.js

```
const mysql = require('mysql2');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
require('dotenv').config();

const db = mysql.createConnection({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
});

db.connect((err) => {
  if (err) console.log('DB Error:', err.message);
  else console.log('DB Connected!');
});
```

```

// REGISTER
exports.register = (req, res) => {
  const { name, email, password } = req.body;
  bcrypt.hash(password, 10, (err, hashed) => {
    if (err) return res.status(500).json({ success: false, message: err.message });
    db.query(
      'INSERT INTO users (name, email, password) VALUES (?, ?, ?)',
      [name, email, hashed],
      (err, result) => {
        if (err) return res.status(500).json({ success: false, message: err.message });
        res.json({ success: true, message: 'Registration successful!' });
      }
    );
  });
};

// LOGIN
exports.login = (req, res) => {
  const { email, password } = req.body;
  db.query('SELECT * FROM users WHERE email = ?', [email], (err, rows) => {
    if (err) return res.status(500).json({ success: false, message: err.message });
    if (rows.length === 0) return res.status(404).json({ message: 'User not found!' });
    bcrypt.compare(password, rows[0].password, (err, valid) => {
      if (!valid) return res.status(401).json({ message: 'Wrong password!' });
      const token = jwt.sign(
        { id: rows[0].id }, process.env.JWT_SECRET, { expiresIn: '7d' }
      );
      res.json({ success: true, token,
        user: { id: rows[0].id, name: rows[0].name, email: rows[0].email }
      });
    });
  });
};

```

File 4: routes/auth.js

```

const express = require('express');
const router = express.Router();
const { register, login } = require('../controllers/authController');

router.post('/register', register);
router.post('/login', login);

module.exports = router;

```

File 5: app.js

```

const express = require('express');
const cors = require('cors');
require('dotenv').config();

const app = express();

app.use(cors({
  origin: '*',
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
}));

```

```

app.use(express.json({ limit: '1mb' }));
app.use('/api/auth', require('./routes/auth'));

app.get('/', (req, res) => {
  res.json({ message: 'API is running!' });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log('Server running on port ' + PORT);
});

```

STEP 6

Run Server with PM2

App: Terminal

Open Terminal in myapp_api folder, then run:

```

cd myapp_api
pm2 start app.js --name "myapp-api" --max-memory-restart 100M
pm2 save
pm2 startup

```

PM2 Daily Commands

Command	What it does
pm2 status	Check if server is running
pm2 start app.js	Start the server
pm2 restart myapp-api	Restart the server
pm2 stop myapp-api	Stop the server
pm2 logs	View error logs

STEP 7

Test in Browser

App: Browser

Type this URL in browser:

```
http://localhost:3000
```

You should see:

```
{ "message": "API is running!" }
```

STEP 8

Connect Flutter to API

App: VS Code

In api_service.dart, set the baseUrl:

```

// For Web Browser
baseUrl: 'http://localhost:3000/api'

// For Android Phone (use your PC's IP address)
// Run ipconfig in CMD to find IPv4 Address
baseUrl: 'http://192.168.X.X:3000/api'

```

PM2 vs node app.js — Key Differences

Feature	node app.js	PM2
Terminal closed	Server stops	Keeps running
PC restarted	Server stops	Auto starts
Memory limit	Cannot set	Can set limit
Error logs	Not available	pm2 logs
Best used for	Development	Production

Follow this guide to build a complete Flutter + Node.js API + MySQL app!